

Relatório Tarefa 8: Coerência de cache e Falso compartilhamento

Aluno: Thiago Jordão Melo da Costa

Data: 16 de junho de 2026

Resumo

Este relatório investiga o impacto de detalhes arquiteturais e de implementação no desempenho de programas paralelos em memória compartilhada utilizando OpenMP. O experimento baseia-se na estimativa do valor de π através do Método de Monte Carlo. Foram implementadas quatro versões do algoritmo para contrastar o uso de variáveis privadas versus vetores compartilhados, bem como o uso de geradores de números aleatórios com estado global (`rand()`) versus geradores reentrantes e seguros para *threads* (`rand_r()`). Os resultados demonstram na prática os gargalos de contenção causados por funções que não são *thread-safe* e o fenômeno de degradação de desempenho conhecido como falso compartilhamento (*false sharing*), inerente aos protocolos de coerência de memória *cache* em processadores *multicore*.

1. Introdução e Fundamentação Teórica

O Método de Monte Carlo para a estimativa de π consiste em sortear aleatoriamente coordenadas (x, y) dentro de um quadrado de lado 2 centrado na origem e contar quantas dessas coordenadas caem dentro do círculo inscrito de raio 1. A razão entre os acertos e o total de tentativas multiplicada por 4 fornece a estimativa de π . Por ser um problema de contagem paralela com iterações independentes, ele é um candidato ideal para o paralelismo.

Contudo, ao paralelizar esse código em arquiteturas de memória compartilhada, dois problemas graves podem surgir e destruir o desempenho computacional:

1. **Segurança de Threads (Thread-Safety):** Funções da biblioteca padrão do C, como o `rand()` e o `strtok`, frequentemente mantêm seu estado interno através de variáveis estáticas, fazendo com que múltiplas *threads* acessem o mesmo estado global simultaneamente. Para evitar a corrupção desses dados, as bibliotecas injetam travas (*locks*) internas, o que força a serialização da execução e destrói o paralelismo.
 2. **Coerência de Cache e Falso Compartilhamento (False Sharing):** A memória *cache* não trafega variáveis individuais da memória RAM, mas sim blocos inteiros e contíguos de memória chamados de **linhas de cache (cache lines)**. Os sistemas utilizam protocolos (como *snooping* ou baseados em diretórios) para garantir que essas linhas se mantenham coerentes entre os núcleos. Se duas *threads* atualizam variáveis diferentes, mas que habitam a mesma linha de *cache* (como índices vizinhos em um vetor), o *hardware* invalida repetidamente a linha inteira nas *caches* adjacentes. Isso força o sistema a buscar os dados na memória RAM constantemente, um problema conhecido como **falso compartilhamento**.
-

2. Implementação (código fonte)

O código em linguagem C a seguir implementa as quatro versões propostas para o experimento, variando o gerador de números aleatórios e a estrutura de dados utilizada para o acúmulo da contagem.

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

#define TOSSES 50000000

int main(int argc, char* argv[]) {
    int thread_count = 4; // Número de threads
    double start, finish;

    omp_set_num_threads(thread_count);
    printf("Simulacao com %d tosses e %d threads.\n\n", TOSSES,
thread_count);

    //
=====
    // VERSÃO 1: rand() + Variável Privada + Critical
    //
=====

    long long global_hits_v1 = 0;
    start = omp_get_wtime();
    #pragma omp parallel
    {
        long long local_hits = 0;
        #pragma omp for
        for (long long i = 0; i < TOSSES; i++) {
            double x = ((double)rand() / RAND_MAX) * 2.0 - 1.0;
            double y = ((double)rand() / RAND_MAX) * 2.0 - 1.0;
            if (x*x + y*y <= 1.0) local_hits++;
        }
        #pragma omp critical
        global_hits_v1 += local_hits;
    }
    finish = omp_get_wtime();
    printf("V1 (rand + critical)      | Pi: %.5f | Tempo: %f s\n",
        4.0 * global_hits_v1 / TOSSES, finish - start);

    //
=====
    // VERSÃO 2: rand() + Vetor Compartilhado (Falso Compartilhamento)
    //
=====

    long long* shared_hits_v2 = (long long*)calloc(thread_count,
sizeof(long long));
    start = omp_get_wtime();
    #pragma omp parallel
    {
        int my_rank = omp_get_thread_num();
        #pragma omp for
        for (long long i = 0; i < TOSSES; i++) {
            double x = ((double)rand() / RAND_MAX) * 2.0 - 1.0;

```

```

        double y = ((double)rand() / RAND_MAX) * 2.0 - 1.0;
        if (x*x + y*y <= 1.0) shared_hits_v2[my_rank]++; // Atualiza
diretamente no vetor
    }
}
long long global_hits_v2 = 0;
for (int i = 0; i < thread_count; i++) global_hits_v2 +=
shared_hits_v2[i];
finish = omp_get_wtime();
printf("V2 (rand + shared array) | Pi: %.5f | Tempo: %f s\n",
    4.0 * global_hits_v2 / TOSSES, finish - start);

//
=====
// VERSÃO 3: rand_r() + Variável Privada + Critical
//
=====

long long global_hits_v3 = 0;
start = omp_get_wtime();
#pragma omp parallel
{
    long long local_hits = 0;
    unsigned int seed = (unsigned int)omp_get_thread_num() +
time(NULL);
    #pragma omp for
    for (long long i = 0; i < TOSSES; i++) {
        double x = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
        double y = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
        if (x*x + y*y <= 1.0) local_hits++;
    }
    #pragma omp critical
    global_hits_v3 += local_hits;
}
finish = omp_get_wtime();
printf("V3 (rand_r + critical) | Pi: %.5f | Tempo: %f s\n",
    4.0 * global_hits_v3 / TOSSES, finish - start);

//
=====
// VERSÃO 4: rand_r() + Vetor Compartilhado (Falso Compartilhamento
isolado)
//
=====

long long* shared_hits_v4 = (long long*)calloc(thread_count,
sizeof(long long));
start = omp_get_wtime();
#pragma omp parallel
{
    int my_rank = omp_get_thread_num();
    unsigned int seed = (unsigned int)my_rank + time(NULL);
    #pragma omp for
    for (long long i = 0; i < TOSSES; i++) {
        double x = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
        double y = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;

```

```
        if (x*x + y*y <= 1.0) shared_hits_v4[my_rank]++; // Falso compartilhamento
    }
}
long long global_hits_v4 = 0;
for (int i = 0; i < thread_count; i++) global_hits_v4 += shared_hits_v4[i];
finish = omp_get_wtime();
printf("V4 (rand_r + shared array) | Pi: %.5f | Tempo: %f s\n",
        4.0 * global_hits_v4 / TOSSES, finish - start);

free(shared_hits_v2);
free(shared_hits_v4);
return 0;
}
```

3. Resultados e Discussão Teórica

Abaixo, os resultados obtidos a partir de uma execução em 4 threads com 50.000.000 de iterações (TOSSES):

Versão	Gerador de RNG	Abordagem de Acúmulo	Tempo de Execução (s)	Valor de π Estimado
V1	rand()	Variável Privada + critical	6.069647	3.14197
V2	rand()	Vetor Compartilhado	4.980800	3.14163
V3	rand_r()	Variável Privada + critical	0.155177	3.14171
V4	rand_r()	Vetor Compartilhado	0.414178	3.14183

O comportamento observado nas execuções fundamenta-se nos princípios de arquitetura de hardware e de desenvolvimento de software paralelo:

Análise das Versões 1 e 2: O Problema do rand() As versões 1 e 2 utilizam a função clássica rand(). Nessas execuções, o desempenho será desastroso (muitas vezes mais lento que uma execução puramente serial). Isso ocorre porque rand() armazena o valor da última semente calculada em uma variável com classe de armazenamento estático, persistindo na memória compartilhada entre as chamadas. Como ela não é thread-safe (não reentrante), a implementação da biblioteca C insere uma trava (lock) de exclusão mútua dentro da função para prevenir a corrupção do gerador. Consequentemente, como as threads passam a maior parte do tempo pedindo números aleatórios, elas ficam enfileiradas aguardando a liberação dessa trava invisível, o que serializa severamente todo o laço paralelo e anula o benefício do multithreading.

Análise da Versão 3: Desempenho Ideal A V3 apresenta o melhor desempenho e escalabilidade do experimento. Ela substitui rand() pela função reentrante rand_r(&seed). A nova função recebe o estado da semente como ponteiro local para a pilha de execução (stack) da thread, eliminando a necessidade de travas da biblioteca padrão. Além disso, a V3 usa uma variável puramente privada (local_hits) alocada em registradores para acumular milhões de contagens internamente. O contato com a memória compartilhada

acontece apenas uma única vez por *thread*, ao término de todo o laço, protegido adequadamente por um `#pragma omp critical`.

Análise da Versão 4: O Efeito do Falso Compartilhamento (*False Sharing*) A V4 resolve o problema do `rand()` utilizando `rand_r()`, mas abandona a variável privada local. Em vez disso, cada *thread* escreve seu acumulador em um índice direto de um vetor compartilhado: `shared_hits_v4[my_rank]++`. Ao medir o tempo, nota-se que a V4 é significativamente mais lenta que a V3. A justificativa está no **falso compartilhamento**. O vetor `shared_hits_v4` possui `thread_count` posições do tipo `long long` (8 bytes cada). Quatro ou oito dessas posições cabem com folga dentro de uma única linha de *cache* (geralmente de 64 bytes). Toda vez que a Thread 0 incrementa o índice 0, a controladora de *cache* do processador demarca **toda a linha de *cache*** como "suja/modificada" e invalida essa linha no núcleo da Thread 1. Quando a Thread 1 tenta atualizar o seu próprio índice 1 no milissegundo seguinte, sofre um *cache miss*, sendo obrigada a parar (*stall*) a CPU e buscar os dados novamente na memória principal. Embora as *threads* não acessem o espaço de memória umas das outras, elas compartilham a mesma linha de *cache* física, gerando um tráfego intenso e redundante no barramento de memória, forçando a CPU a atuar de forma estrangulada.

4. Conclusão

O experimento consolida dois preceitos arquiteturais vitais em computação de alto desempenho em memória compartilhada:

1. Deve-se assegurar rigorosamente que as funções de biblioteca chamadas dentro de regiões paralelas sejam reentrantes e possuam escopo seguro para *threads* (ex: `rand_r` em vez de `rand`), mitigando assim contenções indesejadas.
2. O leiaute das estruturas de memória deve levar em consideração o funcionamento físico da memória *cache*. Estruturas que sofrem múltiplas escritas concorrentes e contínuas por *threads* distintas não podem coexistir densamente empacotadas de forma contígua (falso compartilhamento) sob o risco de drástica penalização de tempo. O padrão ideal é realizar todo acúmulo intensivo localmente em registradores e utilizar sincronização estrita (como redução ou `critical`) para a consolidação final em variáveis compartilhadas.